

# Design of a Configurable Multifunctional Systolic Array Based on FPGA

1<sup>st</sup> Boyao Shi

School of Artificial Intelligence

Beijing University of Posts and Telecommunications

Beijing, China

Email: s1271881297@163.com

2<sup>nd</sup> Shaosheng Li\*

School of Artificial Intelligence

Beijing University of Posts and Telecommunications

Beijing, China

Email: lss@bupt.edu.cn

**Abstract**—In this paper, we propose a configurable universal matrix computation unit based on *FP16*, achieving a unified integrated design for one-dimensional convolution, correlation, and matrix multiplication. The unit employs a systolic array architecture overall, adapting to diverse computational demands through different PE unit structures and configurable input matrix or vector dimensions. This approach maximizes resource efficiency while maintaining accuracy and throughput. Simulation results demonstrate that the designed unit can perform correlation, convolution, and matrix multiplication operations.

**Keywords**—Floating-point operations, configurable, matrix operations, systolic array

## I. INTRODUCTION

Floating-point representation is a numerical representation widely used in computing across fields such as computer science and communications. Based on the IEEE 754 standard, floating-point numbers divide the binary 0-1 sequence into a combination of sign bits (Sign), exponent bits (Exponent), and mantissa bits (Mantissa/Fraction) to represent real numbers. The conversion relationship between floating-point representation and decimal real numbers is as follows, where  $D$  denotes the sign representation of the decimal real number,  $S$  represents the sign bit,  $M$  denotes the mantissa bits, and  $E$  represents the exponent bits:

$$D = (-1)^S \times 1.M \times 2^{E-Bias} \quad (1)$$

To facilitate comparison operations and avoid storing negative exponents explicitly, IEEE 754 employs a biased exponent representation. The bias value is determined by the number of exponent bits and is calculated using the following formula, where  $k$  denotes the number of exponent bits in the floating-point format:

$$Bias = 2^{k-1} - 1 \quad (2)$$

Common floating-point formats include 64-bit double-precision floating-point, 32-bit single-precision floating-point, 16-bit half-precision floating-point, and the newly proposed UE8M0, among others. Taking half-precision floating-point numbers (Half Float Precision 16, *FP16*) as an example, within a 16-bit width: the 15th bit is the sign bit, bits 14 to 10 represent the exponent, and bits 9 to 0 represent the mantissa. Its structure can be represented as shown in Figure 1. UE8M0 is a newly proposed floating-point format tailored for AI training. While

not classified under the IEEE 754 standard, it maximizes dynamic range, reduces memory footprint, and enhances inference and throughput in AI training by eliminating the sign bit and sacrificing precision.

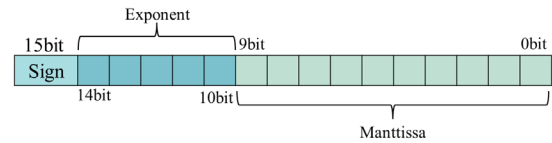


Fig. 1. Structural Diagram of *FP16*.

Contemporary research on floating-point arithmetic has expanded significantly beyond AI model inference applications, with an increasing number of researchers integrating floating-point operations into hardware design to develop specialized accelerators. The multiply-accumulate (MAC) operation has emerged as a focal point of investigation. Reference [1] employs a divide-and-conquer approach combined with the Karatsuba-Ofman algorithm, recursively decomposing multiplication into smaller partial products to implement and comprehensively verify LUT-based single-precision, double-precision, and quadruple-precision floating-point multipliers. Reference [2] proposes an approximate multiplier utilizing LUT sharing and carry exchange techniques. Reference [3] presents a floating-point multiplication unit employing radix-4 Booth encoding, hierarchical compressors, and Wallace trees. Reference [4] implements a floating-point MAC unit based on radix-4 Booth encoding and accumulator architecture.

In reviewing research on floating-point MAC operations, Booth encoding has been extensively adopted, notably in [3] and [4], with numerous subsequent studies building upon this approach. Reference [5] simplifies complex multiplication by sharing the Booth encoder to reduce hardware overhead. Reference [6] proposes an approximate octal Booth multiplier for Sobel edge detection applications. Reference [7] designs an energy-efficient approximate soft-core Booth multiplier based on adaptive logic modules (ALMs) to enhance performance in resource-constrained embedded image processing and machine learning applications on FPGAs.

The systolic array is a classical in-memory computing architecture widely employed for hardware-accelerated matrix multiplication. Reference [8] proposes a vector systolic accelerator incorporating a bit-partitioned multi-precision floating-point processing unit, which achieves high hardware

utilization and state-of-the-art energy efficiency across *FP16*, *FP32*, and *FP64* operations. Meanwhile, [9] introduces a skewed pipeline architecture for systolic arrays that utilizes a speculative exponent forwarding mechanism to enable parallel execution across consecutive processing elements, significantly reducing latency and improving energy efficiency for reduced-precision floating-point arithmetic in deep learning applications. Extending these developments, [10] proposes a centralized systolic array architecture for sparse matrix multiplication, maintaining low area and power overhead while achieving superior efficiency in sparse acceleration scenarios prevalent in modern neural networks. Complementing this, [11] presents a lookup table (LUT)-based systolic array that enhances throughput through vectorization and resource reuse, addressing computational bottlenecks in dense matrix operations. Furthermore, [12] introduces PoMX, a multi-mode systolic array matrix engine that coordinates multiple small systolic arrays operating in parallel, effectively reducing data transfer latency and improving resource efficiency for dynamic workloads. Similarly, [13] proposed an efficient FPGA-based DETR accelerator leveraging dual systolic arrays, which significantly enhances parallel efficiency and scalability while effectively mitigating the latency bottleneck in nonlinear computations. Collectively, these systolic array architectures have been widely employed in neural network acceleration, consistently demonstrating enhanced computational efficiency, reduced energy consumption, and improved performance scalability. Building upon coarse-grained reconfigurable array (CGRA) architectures and dynamic scheduling mechanisms, [14] proposes a reconfigurable stream data processor (RSDP) that significantly enhances performance for stream processing tasks.

However, traditional systolic arrays often feature a fixed structure, which limits their efficiency and flexibility when handling problems exceeding their physical scale or being applied to scenarios beyond their original design objectives. Furthermore, there is a notable lack of systolic array designs capable of generalizing effectively across diverse application scenarios.

This work presents an enhanced processing element (PE) architecture for systolic arrays leveraging *FP16* floating-point precision, which seamlessly integrates artificial intelligence and communication application scenarios to realize a general-purpose reconfigurable matrix processing architecture on FPGA platforms. Accordingly, we propose a versatile *FP16*-based computational framework that natively supports one-dimensional convolution/correlation operations, and matrix multiplication with fully configurable input dimensions, thereby substantially improving operational flexibility and extending applicability to a wide range of computational workloads. The primary contributions of this work are as follows:

- Design of an optimized *FP16* MAC unit combining radix-4 Booth encoding with Wallace tree compression to simplify multiplication logic, supporting both real and complex number operations while maintaining computational accuracy and minimizing hardware resource consumption.
- Implementation of a configurable general-purpose processing element (GPE) supporting real and complex

number multiply-accumulate operations in multiple operational modes.

- Development of a reconfigurable systolic array architecture supporting real and complex matrix operations, including convolution, correlation, and matrix multiplication.

The remainder of this paper is organized as follows: Section II introduces the fundamental concepts and architectural foundations. Section III details the *FP16* arithmetic implementation, the enhanced processing element design, and the reconfigurable systolic array architecture. Section IV presents the hardware implementation and performance analysis. Section V concludes the paper and discusses future research directions.

## II. PROPOSED DESIGN

General Processing Array (GPA) is constructed based on General Processing Elements (GPEs) designed for matrix multiplication and one-dimensional convolution operations. GPEs can decode control mode parameters and instantiate themselves to form structures optimized for either vector or matrix operations.

### A. GPE Unit Design

The GPE unit is a configurable processing element that supports four operational modes through the *PE\_MODE* parameter. The mode decoding and selection logic is illustrated in Figure 2. In matrix multiplication mode, two matrices enter the systolic array at different positions with varying latencies, with data flowing from the upper-left corner to the lower-right corner. The GPE unit receives data from matrix A and matrix B, which are latched by input registers before being fed into the computational unit. For complex matrices (32-bit mode), the unit instantiates four *FP16* multipliers to perform complex multiplication, with the real and imaginary components computed by two separate *FP16* adders, respectively. For real-valued matrices (16-bit mode), a single *FP16* multiplier is employed. Each multiplication result is accumulated with the internal accumulator via an *FP16* adder, implementing the MAC functionality. Data propagates rightward and downward under register control. Upon completion of the entire data flow, the accumulated result in each GPE unit corresponds to the element at the respective position in the matrix product.

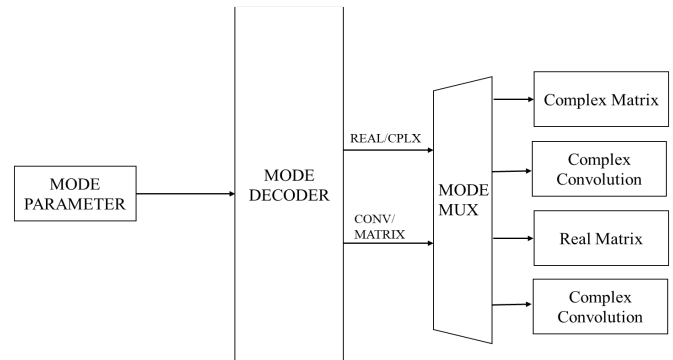


Fig. 2. Mode Decoding and Selection.

In one-dimensional convolution mode, the computation proceeds in two distinct phases. During the weight loading phase, when the loading flag is deasserted and the enable signal is active, weight data is stored in dedicated weight registers while partial sum registers are cleared; no valid output is generated at this stage. Once loading is complete, the loading flag is asserted, initiating the computation phase. The GPE unit multiplies the stored weights with incoming signal data, and the result is summed with the partial product from the preceding processing element via the adder. A key design feature lies in the multiplier input selection logic, which automatically switches based on the mode decoding parameter. In convolution mode, the stored weight value is selected as one multiplier input, while the accumulator receives the partial sum from the previous stage rather than the internal accumulation register. The computation result is directly output as the final result and simultaneously passed to the subsequent GPE unit as a partial product. Through this approach, only the multiply-accumulate results propagate through the systolic array while weights remain stationary within each processing element. The convolution result is obtained at the output of the final GPE unit after excluding several invalid outputs at the beginning of the sequence.

### B. GPA Design

The GPA is a systolic array architecture composed of multiple GPE units, supporting both matrix multiplication and one-dimensional convolution modes through the PE\_MODE parameter. The computation workflow encompasses data input, processing, and output operations in a continuous pipeline.

The module receives external input data and stores it in internal distributed RAM. In matrix mode, the first matrix is stored row-wise with each row containing a specified number of elements, while the second matrix is stored column-wise with each column containing the corresponding number of elements. In convolution mode, signal data is stored as complete sequences containing all samples, and convolution kernel weights are stored sequentially. An input controller tracks data reception progress through counters. Upon completion of receiving all required rows and columns in matrix mode, or all signal channels and weights in convolution mode, a completion flag is asserted to initiate computation.

The computational structures for matrix and convolution modes are illustrated in Figure 3. In matrix mode, the two matrices enter the two-dimensional systolic array at different positions with varying latencies. Data from the first matrix flows horizontally from left to right, while data from the second matrix flows vertically from top to bottom. GPE units are instantiated according to their row and column positions, with each processing element receiving data from its left neighbor and top neighbor. After performing multiply-accumulate operations, each unit passes the first matrix data rightward and the second matrix data downward while continuously accumulating results in its internal accumulator. After a specified number of computation cycles, the accumulator in each processing element contains the final result for the corresponding position in the matrix product.

In convolution mode, the GPE units are arranged in a chain configuration, with weights loaded into each processing element's weight register during the initial cycle. A critical

design feature is the hierarchical delay broadcast mechanism: signal data is broadcast to all GPE, but with different reception delays—the first unit receives data without delay, while each subsequent unit experiences an increasing delay of one cycles, ensuring proper alignment of the convolution window. Each GPE outputs its multiply-accumulate result, which passes through a register delay of one cycle before serving as the accumulation input for the downstream unit, forming a register-delayed accumulation chain. Signal data is broadcast sequentially over multiple cycles, followed by zero inputs to flush the accumulation chain. The output of the final GPE becomes valid after a calculated number of cycles, and an output buffer continuously collects the results.

Upon assertion of the computation completion flag, result collection logic transfers GPE outputs to the result storage array. In matrix mode, accumulated results from each unit are written to a matrix result storage array, while in convolution mode, samples are read from the output buffer and written to a convolution result storage array. An output controller subsequently delivers results through the output port in sequence—matrix mode outputs elements in row-major order, while convolution mode outputs all computed samples. A valid output signal remains high during the output period and is cleared upon completion, awaiting the next computation cycle.

### C. Key Parameter to Calculation

The architectural design of GPA is based on four key parameters:  $C_1$ ,  $R_1$ ,  $C_2$ , and  $R_2$ . In matrix operation mode,  $C_1$  and  $R_1$  represent the number of columns and rows of matrix A, respectively, while  $C_2$  and  $R_2$  represent the number of columns and rows of matrix B, respectively, with the constraint condition  $C_1 = R_2$ . Consequently, the number of processing elements (GPEs) constituting the GPA in matrix mode, denoted as  $N_{M-GPE}$ , can be expressed as (3).

$$N_{M-GPE} = R_1 \times C_2 \quad (3)$$

The computation cycle of GPA in matrix mode,  $T_{M-GPA}$ , can be formulated as (4).

$$T_{M-GPA} = R_1 + R_2 + C_2 - 2 \quad (4)$$

In convolution operation mode and correlation operation mode, the dimensional parameters  $R_1$  and  $C_2$  are both set to 1, where  $R_2$  represents the input signal length and  $C_1$  represents the kernel length. Therefore, the number of GPEs constituting the GPA in convolution mode,  $N_{C-GPE}$ , can be expressed as (5).

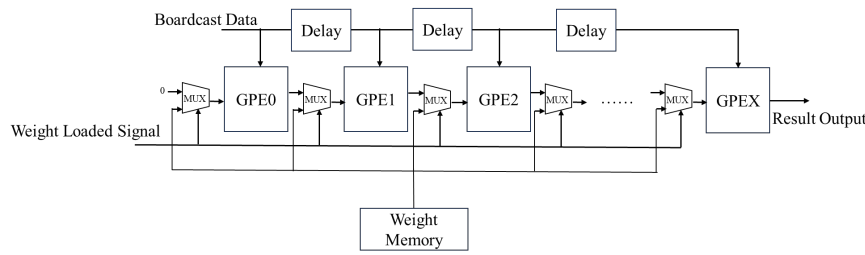
$$N_{C-GPE} = C_1 \quad (5)$$

The computation cycle of GPA in convolution mode,  $T_{C-GPA}$ , can be formulated as (6).

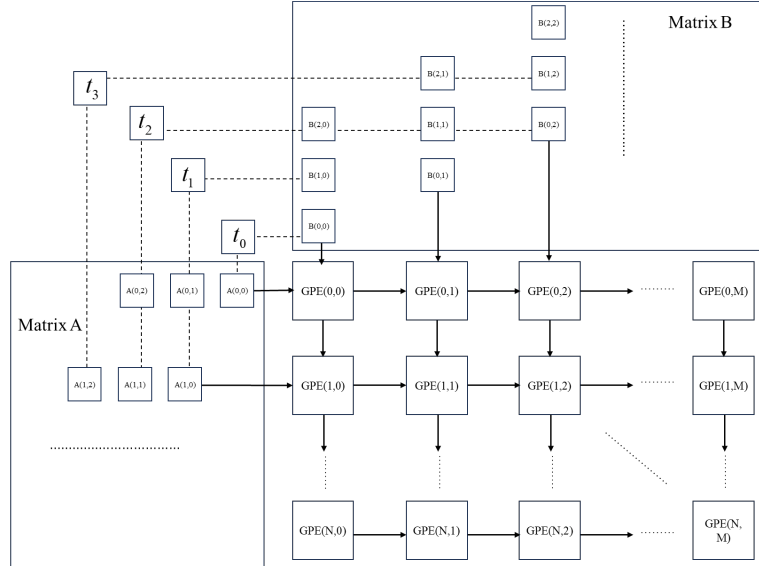
$$T_{C-GPA} = R_2 + 3 \times C_1 - 1 \quad (6)$$

### D. FP16-Based Floating-Point Arithmetic

The designed systolic array supports both real and complex number operations using FP16 format. Real numbers utilize



(a) 1D convolution with broadcast mechanism.



(b) 2D matrix multiplication with bidirectional data flow.

Fig. 3. Reconfigurable GPA architectures for convolution and matrix operations.

standard 16-bit representation, while complex numbers employ 32-bit encoding with the upper and lower 16 bits representing the real and imaginary components, respectively. Real multiply-accumulate operations are executed directly, whereas complex multiplication employs four real multipliers and two adders, with complex addition computed component-wise.

For *FP16* multiplication, the exponent undergoes bias-adjusted addition/subtraction based on sign bits. To reduce computational complexity, mantissa multiplication utilizes radix-4 Booth encoding with Wallace tree compression, governed by the encoding rules in Table I. The Booth selectors generate partial products from 3-bit multiplier segments, producing a 26-bit intermediate result after compression. A leading zero counter determines the shift magnitude, followed by rounding per IEEE 754 "round to nearest even" standard. The output stage handles special cases including infinity and overflow.

Given the similar computational processes for addition and subtraction, a unified calculation module is designed to enhance code reusability and reduce resource consumption. Upon receiving two *FP16* inputs, the module first decodes the inputs to extract the sign, exponent, and mantissa, while detecting special values (zero, infinity, NaN, and subnormal numbers). For subtraction, the sign of the second operand is inverted. Normal numbers have an implicit leading bit of 1 appended, while subnormal numbers utilize a fixed minimum exponent. The operands are then compared to determine their relative magnitudes, and the exponent difference is calculated. The mantissa of the smaller operand is right-shifted for alignment (extended to 27 bits to preserve precision), with exponent differences exceeding 16 treated as zero. Based on the sign bits, the effective operation type is determined, and both 28-bit addition and subtraction are computed in parallel before selecting the correct output.

TABLE I. RADIX-4 BOOTH ENCODING RULES.

| $Y_{i+1}$ | $Y_i$ | $Y_{i-1}$ | Encoding | Operation     | Partial Product | $Y_{i+1}$ | $Y_i$ | $Y_{i-1}$ | Encoding | Operation     | Partial Product |
|-----------|-------|-----------|----------|---------------|-----------------|-----------|-------|-----------|----------|---------------|-----------------|
| 0         | 0     | 0         | 0        | $0 \times M$  | 0               | 1         | 0     | 0         | -2       | $-2 \times M$ | $-2M$           |
| 0         | 0     | 1         | +1       | $+1 \times M$ | $+M$            | 1         | 0     | 1         | -1       | $-1 \times M$ | $-M$            |
| 0         | 1     | 0         | +1       | $+1 \times M$ | $+M$            | 1         | 1     | 0         | -1       | $-1 \times M$ | $-M$            |
| 0         | 1     | 1         | +2       | $+2 \times M$ | $+2M$           | 1         | 1     | 1         | 0        | $0 \times M$  | 0               |

TABLE II. RESOURCE CONSUMPTION ANALYSIS OF GPE UNIT CONFIGURATIONS IN REAL VALUE MODE.

| Logic Utilization  | Real Mode   |             |        |             | Logic Utilization           | Real Mode   |             |        |             |
|--------------------|-------------|-------------|--------|-------------|-----------------------------|-------------|-------------|--------|-------------|
|                    | Convolution | Utilization | Matrix | Utilization |                             | Convolution | Utilization | Matrix | Utilization |
| CLB LUTs (1728000) | 912         | 0.05        | 860    | 0.05        | GLOBAL CLOCK BUFFERs (1344) | 1           | 0.07        | 1      | 0.07        |
| CLB REG (3456000)  | 706         | 0.02        | 720    | 0.02        | Frequency (MHz)             | 500         | /           | 500    | /           |
| CARRY8 (216000)    | 10          | /           | 10     | /           | Power (mW)                  | 53          | /           | 47     | /           |
| F7 Muxes (864000)  | 1           | /           | 1      | /           | Throughput (Gop/s)          | 2           | /           | 2      | /           |
| Bonded IOB(702)    | 83          | 11.82       | 84     | 11.82       | Delay(ns)                   | 32          | /           | 30     | /           |

TABLE III. RESOURCE CONSUMPTION ANALYSIS OF GPE UNIT CONFIGURATIONS IN COMPLEX VALUE MODE.

| Logic Utilization  | Complex Mode |             |        |             | Logic Utilization          | Complex Mode |             |        |             |
|--------------------|--------------|-------------|--------|-------------|----------------------------|--------------|-------------|--------|-------------|
|                    | Convolution  | Utilization | Matrix | Utilization |                            | Convolution  | Utilization | Matrix | Utilization |
| CLB LUTs (1728000) | 3473         | 0.2%        | 3375   | 0.2%        | GLOBAL CLOCK BUFFERs(1344) | 1            | 0.07%       | 1      | 0.07%       |
| CLB REG (3456000)  | 2617         | 0.08%       | 2647   | 0.08%       | Frequency(MHz)             | 500          | /           | 500    | /           |
| CARRY8 (216000)    | 40           | /           | 40     | /           | Power(mW)                  | 136          | /           | 123    | /           |
| F7 Muxes (864000)  | 4            | /           | 4      | /           | Throughput(GFLOPS)         | 8            | /           | 8      | /           |
| Bonded IOB(702)    | 163          | 23.22%      | 163    | 23.22%      | Delay(ns)                  | 44           | /           | 42     | /           |

Following normalization, IEEE 754 rounding is executed by extracting the guard bit, round bit, and sticky bit. The "round to nearest even" rule determines whether to increment the result, and any overflow from rounding necessitates further exponent adjustment. If the exponent falls to zero or becomes negative, subnormal number processing is invoked, right-shifting the mantissa and setting the exponent to zero. Finally, exceptions (invalid operations, overflow) are detected, and the output result is assembled according to priority.

### III. EXPERIMENTAL RESULTS

The GPE unit and GPA structure are described using Verilog HDL and prototyped on the Xilinx UltraScale+ XCVU13P-FHGB2104-2-I platform. The development was conducted using Xilinx Vivado 2023.1. Since the design does not rely on any IP cores, it exhibits excellent portability and can be readily migrated to different FPGA devices from various vendors.

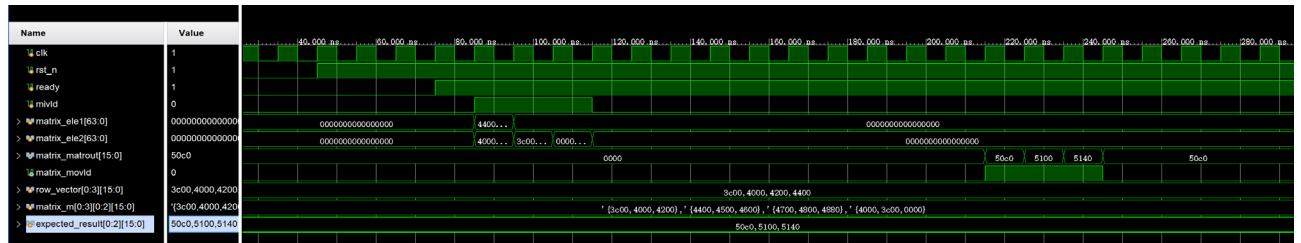
#### A. GPE Unit Results and Analysis

Tables II and III present the resource utilization, operating frequency, and dynamic power consumption of the GPE unit

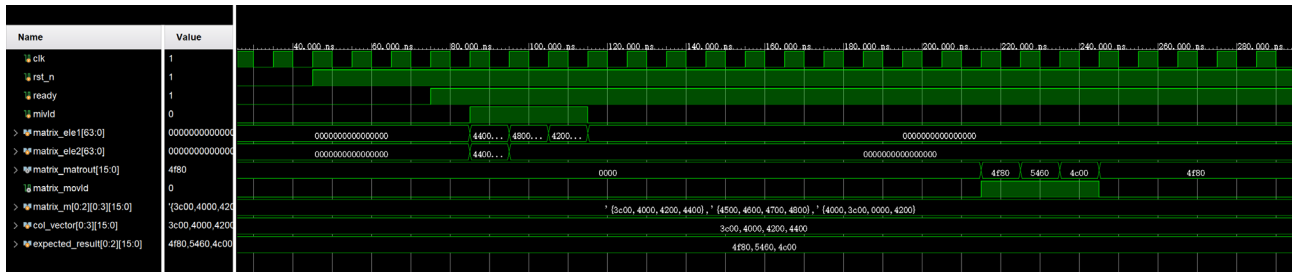
across four operational modes. Convolution mode exhibits higher resource consumption than matrix mode due to additional control logic and weight storage requirements. Complex mode consumes approximately four times the resources of real mode, as each complex MAC operation requires four multiplications and four additions. Notably, dynamic power scales sub-linearly with resource utilization, attributed to shared clock infrastructure and non-uniform register switching activity across clock cycles.

#### B. GPA Results and Analysis

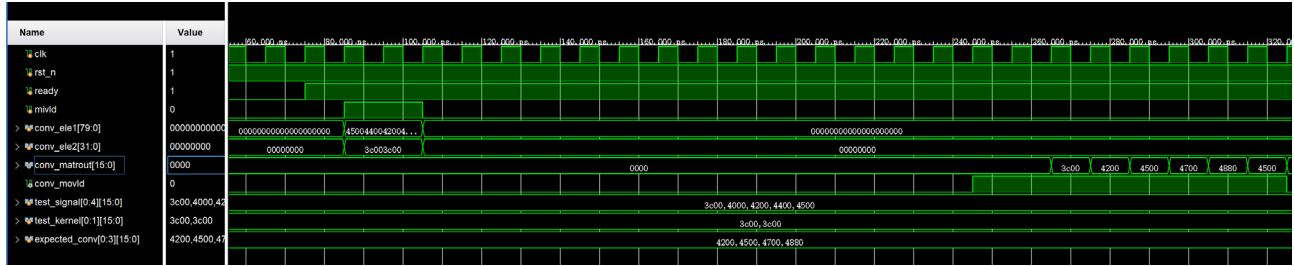
Compared to the GPE unit that implements basic multiply-accumulate functionality, the GPA is designed for larger-scale computations. Due to the diverse reconfiguration possibilities inherent in the architecture, it is impractical to enumerate synthesis results for every possible configuration. To validate the correctness of the design, comprehensive simulations were conducted, and representative simulation results are presented herein.



(a) Real-valued matrix-column vector multiplication.



(b) Real-valued row vector-matrix multiplication.

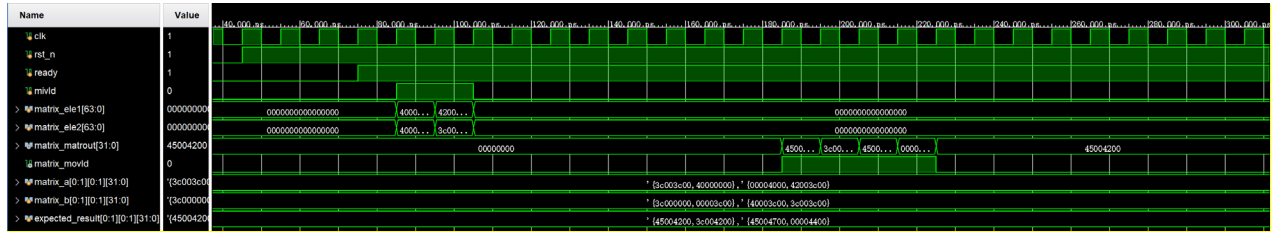


(c) Real-valued convolution.

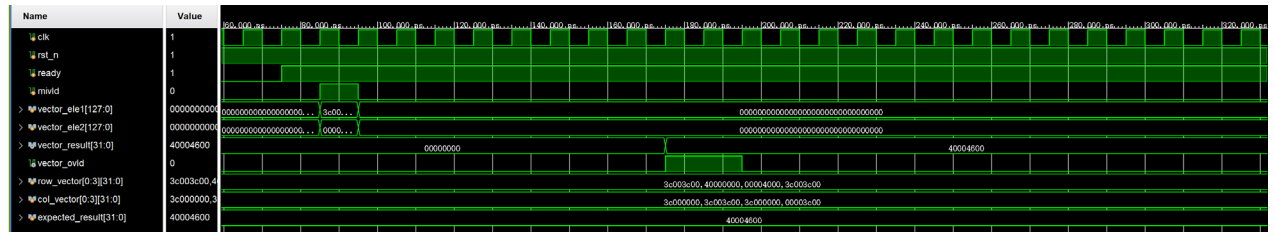
Fig. 4. Simulation Results in Real-Valued Mode.

Figures 4 and 5 present simulation results for real-valued and complex-valued modes, respectively. The architecture was verified through four key operations—convolution, matrix-

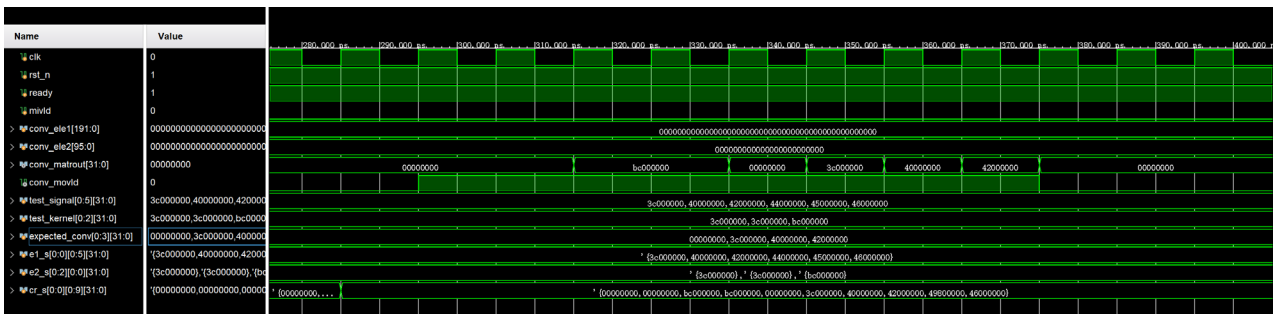
matrix, matrix-vector, and vector-vector multiplication—using randomly generated test cases. Validation confirmed correct operation in both modes.



(a) Complex-valued matrix-matrix multiplication



(b) Complex-valued vector-vector multiplication



(c) Complex-valued convolution

Fig. 5. Simulation Results in Complex-Valued Mode.

### C. Comparison with Related Work

In Table IV, we present a comparative analysis of GPA against related state-of-the-art designs. Given the inherent variability introduced by configurable operation modes, we focus our comparison on a single GPA instance evaluated across representative computational scenarios. As our primary design objective emphasizes architectural generality rather than mode-specific optimization, the comparison results demonstrate that GPA achieves notably low latency across diverse workloads.

To maximize configurability and ensure portability across heterogeneous FPGA platforms with varying resource availability, we deliberately eschew DSP blocks in favor of a LUT- and register-based implementation. This design choice yields reduced power consumption at the expense of peak throughput compared to DSP-centric architectures. The design in [11] employs a lookup table (LUT)-based approach with pre-computed results and operates on narrower fixed-point data types, thereby achieving higher throughput through simplified arithmetic operations. In contrast, GPA implements a fully pipelined floating-point datapath with enhanced precision. While this results in lower raw throughput relative to [11], GPA operates at a comparable clock frequency and, critically, provides IEEE 754-compliant FP16 computation capabilities that are fundamentally incompatible with the table-lookup paradigm of [11].

The architecture presented in [14] adopts a runtime reconfigurable approach, leveraging data-driven asynchronous triggers to achieve rapid task switching and employing configuration table pre-loading with distributed Processing Element Array (PEA) controllers to minimize reconfiguration overhead. This design excels in multi-task streaming scenarios with high system efficiency. GPA, by contrast, utilizes a

standard systolic array topology with compile-time parameterization, offering lower latency for single-task execution and a substantially simpler hardware implementation compared to [14]. Both architectures support FP16 precision with comparable computational capabilities. However, the runtime reconfiguration mechanism in [14] introduces significant architectural complexity, and its advantages are less pronounced in applications without frequent task switching requirements.

In summary, through its parameterized systolic array architecture, GPA achieves an effective balance between low latency, reduced power consumption, complete FP16 floating-point support, and multi-mode configurability. Compared to domain-optimized designs, GPA offers distinct advantages in terms of generality, flexibility, and design simplicity.

### IV. CONCLUSION

This paper presents a configurable systolic array architecture using *FP16* floating-point arithmetic for matrix multiplication and one-dimensional convolution. The hierarchical design, comprising GPE units and GPA structure with optimized *FP16* arithmetic, efficiently operates in resource-constrained environments. Applications include digital signal processing, neural network acceleration, and scientific computing. Future work will extend precision format support, implement dynamic power management, and optimize area-performance trade-offs for higher throughput in domain-specific applications.

### ACKNOWLEDGMENT

This work was supported by the National Major Scientific Instruments and Equipments Development Project of National Natural Science Foundation of China (62427810).

TABLE IV. COMPARISON OF THE PROPOSED GPA WITH EXISTING DESIGNS

|                         | <i>Supported Precision(s)</i> | <i>Size</i>           | <i>LUT</i> | <i>REG</i> | <i>Frequency(MHz)</i> | <i>Throughput(GOP/s)</i> | <i>Latency(cycle)</i> | <i>Power (W)</i> |
|-------------------------|-------------------------------|-----------------------|------------|------------|-----------------------|--------------------------|-----------------------|------------------|
| Proposed GPA            | <i>FP16</i>                   | 3×3 real value matrix | 7968       | 6850       | 500                   | 9                        | 28                    | 3.104            |
|                         |                               | 5×5 real value matrix | 22039      | 18922      |                       | 25                       | 52                    | 3.126            |
| [14]                    | Int16, <i>FP16</i>            | 32×32 matrix          | N/A        | N/A        | 600                   | 644(GMAC/s)              | 257                   | N/A              |
| [11]<br>(NVSA LBD/tile) | <i>Int8, Int4</i>             | 5×5 matrix            | About 1000 | N/A        | 515                   | 25.75 (estimated)        | 21                    | 0.6(estimated)   |

<sup>a</sup> Throughput refers to the peak throughput.

### REFERENCES

- [1] N. Chabini, M. C. Wolf and R. Beguenane, "LUT-Based Multipliers for IEEE-754 Floating Point Arithmetic on FPGAs," 2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), Yorktown Heights, NY, USA, 2024, pp. 698-701.
- [2] Y. Guo, Q. Zhou, X. Chen and H. Sun, "Hardware-Efficient Multipliers With FPGA-Based Approximation for Error-Resilient Applications," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 71, no. 12, pp. 5919-5930, Dec. 2024.
- [3] D. Vaithyanathan, J. B. Pari, S. Keerthana and K. Bharathan, "Implementation of Floating Point Unit based on Booth Multiplier and Compressor Adder," 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT), Coimbatore, India, 2019, pp.
- [4] B. Zhou, G. Wang, G. Jie, Q. Liu and Z. Wang, "A High-Speed Floating-Point Multiply-Accumulator Based on FPGAs," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 29, no. 10, pp. 1782-1789, Oct. 2021.
- [5] A. Kali, B. S. Khuntia, S. L. Sabat and P. K. Meher, "A Low-Complexity Design for Complex Multiplication Using Radix-4 Booth Encoding," 2024 IEEE 4th International Conference on VLSI Systems, Architecture,



- Technology and Applications (VLSI SATA), Bangalore, India, 2024, pp. 1-5.
- [6] R. Orugu, S. Padamata, Y. Kollati, L. Nakka, Y. Nunna and R. Sai Midhilesh Mamidi, "FPGA Design and Implementation of Approximate Radix-8 Booth Multiplier," 2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE), Ballari, India, 2024, pp.
  - [7] Z. Aizaz, K. Khare and A. Tirmizi, "FASBM: FPGA-specific Approximate Sum-based Booth multipliers for energy efficient Hardware Acceleration of Image Processing and Machine Learning Applications," 2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Marina Del Rey, CA, USA, 2023, pp. 210-210.
  - [8] K. Li et al., "A Vector Systolic Accelerator for Multi-Precision Floating-Point High-Performance Computing," 2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS), Incheon, Korea, Republic of, 2022, pp. 226-229.
  - [9] D. Filippas, C. Peltekis, G. Dimitrakopoulos and C. Nicopoulos, "Reduced-Precision Floating-Point Arithmetic in Systolic Arrays with Skewed Pipelines," 2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS), Hangzhou, China, 2023, pp. 1-5.
  - [10] Z. Wang, R. Sun, X. He, T. Ma and A. Zou, "DenSparSA: A Balanced Systolic Array Approach for Dense and Sparse Matrix Multiplication," 2025 62nd ACM/IEEE Design Automation Conference (DAC), San Francisco, CA, USA, 2025, pp. 1-7.
  - [11] A. K. Tiwary, S. Gajawada, J. Shah and N. Rao, "LUTAccel: Look-up-Table Based Vector Systolic Accelerator on FPGAs," 2025 26th International Symposium on Quality Electronic Design (ISQED), San Francisco, CA, USA, 2025, pp. 1-7.
  - [12] J. Li, J. Zhang, J. Zhang, Z. Zheng, H. Guo and L. Huang, "Efficient Matrix Engine Design Based on Collaborative Small-Scale Systolic Arrays," 2025 2nd International Conference on Electronic Engineering and Information Systems (EEISS), Nanjing, China, 2025, pp. 1-6.
  - [13] A. Chen, R. Li, P. Qiao and Y. Shao, "FPGA-Based High-Throughput Accelerator for DETR Detection Algorithm," 2025 5th International Symposium on Computer Technology and Information Science (ISCTIS), Xi'an, China, 2025, pp. 1014-1020.
  - [14] R. Qiao and Z. Bu, "Reconfigurable Technology in Streaming Data Processor," 2025 10th International Conference on Electronic Technology and Information Science (ICETIS), Hangzhou, China, 2025, pp. 109-114.